ELEC60004
EE9-AML2-07

IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

EEE/EIE PART III/IV: MEng, BEng and ACGI

MACHINE REASONING

Monday, 1 December 2025, 10:00 AM

Time allowed: 3:00 hours

There are FIVE questions on this paper.

Answer FOUR questions.

All questions carry equal marks.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : J. Pitt
Second Marker(s) : C. Qin

© Imperial College London

ELEC60004/70068. MACHINE REASONING

Information for Students

KE Calculus

α-Formula Rules

$$\frac{p \wedge q}{\begin{array}{c} p \\ q \end{array}} \qquad \frac{\neg(p \vee q)}{\begin{array}{c} \neg p \\ \neg q \end{array}} \qquad \frac{\neg(p \rightarrow q)}{\begin{array}{c} p \\ \neg q \end{array}}$$

β-Formula Rules

$$\frac{p \rightarrow q}{\begin{array}{l} p \\ \quad q \end{array}} \qquad \frac{p \rightarrow q}{\begin{array}{l} \quad \neg q \\ \neg p \end{array}} \qquad \frac{p \vee q}{\begin{array}{l} \neg p \\ \quad q \end{array}} \qquad \frac{p \vee q}{\begin{array}{l} \quad \neg q \\ p \end{array}} \qquad \frac{\neg(p \wedge q)}{\begin{array}{l} p \\ \quad \neg q \end{array}} \qquad \frac{\neg(p \wedge q)}{\begin{array}{l} \quad q \\ \neg p \end{array}}$$

Double Negation Elimination

$$\frac{\neg(\neg p)}{p}$$

η-Formula Rules

$$\frac{p \leftrightarrow q}{\begin{array}{l} p \\ \quad q \end{array}} \qquad \frac{p \leftrightarrow q}{\begin{array}{l} \neg p \\ \quad \neg q \end{array}} \qquad \frac{p \leftrightarrow q}{\begin{array}{l} \quad q \\ p \end{array}} \qquad \frac{p \leftrightarrow q}{\begin{array}{l} \quad \neg q \\ \neg p \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} p \\ \quad \neg q \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} \quad q \\ \neg p \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} \neg p \\ \quad q \end{array}} \qquad \frac{\neg(p \leftrightarrow q)}{\begin{array}{l} \quad \neg q \\ p \end{array}}$$

Branching Rule (PB)

$$\begin{array}{ccc} & \diagup\diagdown & \\ p & & \neg p \end{array}$$

Closure Rule

$$\frac{\begin{array}{c} p \\ \neg p \end{array}}{\times}$$

γ and δ Formula Rules (Predicate Logic)

γ rule	$\dfrac{\forall x.A}{A_{\{c/x\}}}$	$\dfrac{\neg\exists x.A}{\neg A_{\{c/x\}}}$	for any constant c
δ rule	$\dfrac{\exists x.A}{A_{\{c/x\}}}$	$\dfrac{\neg\forall x.A}{\neg A_{\{c/x\}}}$	for a constant c new to the branch

Modal Formula Rules (Modal Logics K and S5)

K

$\Box$-rule	$\dfrac{\sigma : \Box p}{\sigma n : p}$	$\dfrac{\sigma : \neg\Diamond p}{\sigma n : \neg p}$	for any available σn
$\Diamond$-rule	$\dfrac{\sigma : \Diamond p}{\sigma n : p}$	$\dfrac{\sigma : \neg\Box p}{\sigma n : \neg p}$	for σn unrestricted

S5

$\Box$-rule	$\dfrac{i : \Box p}{j : p}$	$\dfrac{i : \neg\Diamond p}{j : \neg p}$	for any available integer j
$\Diamond$-rule	$\dfrac{i : \Diamond p}{j : p}$	$\dfrac{i : \neg\Box p}{j : \neg p}$	for j unrestricted

1. This question is about search space formalisation and the General Graph Search Engine (GGSE).

a) In the context of problem-solving search using the GGSE, explain the difference between a *state* and a *node*, and define how a *state*, a *node*, a *path* and a *graph* can be represented in Prolog, so that it can be used with the GGSE.

[6]

b) The Blocks World scenario imagines a set of movable coloured blocks situated on a table, as illustrated in Figure 1.1.

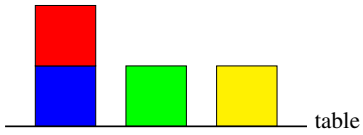


Figure 1.1 Blocks World: Initial State

There is a robot which can perform two actions: it can *stack* a block by taking any single block that is on the table and putting it on any other block, and it can *unstack* any block that is on another block. The goal is to build stacks of blocks called *towers* in a specified arrangement of colours.

Define a state space formulation for the Blocks World in Prolog, and give an example using Figure 1.1.

Explain why your formulation does (or does not) include explicit representation of the table.

[4]

c) Write state-change rules for the two robot actions, *stack* and *unstack*.

Explain why, although there could be multiple *stack* and *unstack* actions that the robot could perform in any given state, only two state-change rules are needed by the GGSE for this problem.

Explain clearly how unification and/or the conditions on the state-change rules are used to determine which of the two actions can be (validly) performed (i.e., how the conditions distinguish between a single block and a tower).

[10]

2. This question is about search space formalisation, the General Graph Search Engine (GGSE) and search algorithms.

a) Explain the difference between Uniform Cost, Best First and A^* search algorithms with respect to the order in which they test and expand paths.

Compare and contrast the three algorithms with respect to appropriate criteria.

[6]

b) Give the two conditions that a heuristic h must satisfy if it is to be used in the A^* search algorithm.

Explain why these two conditions must be satisfied.

Explain why these conditions might make one heuristic perform "better" than another in A^* search.

[6]

c) The Farmer-Wolf-Goat-Cabbage (FWGC) problem used a state space formulation that represented the domain objects according to their locations. The Castle Vanya problem used a state space formulation that represented the domain objects according to their crossing times, and the locations as two lists.

Reformulate the FWGC problem, but now representing each domain object as a term and distinguishing between left and right hand banks. Give the initial state, the goal state, and specify at most **two** state change rules.

[8]

3. This question is about knowledge representation and reasoning with logic and Prolog.

a) Explain what it means for a logical system (e.g. propositional logic, predicate logic) to be decidable.

Explain what it means for an algorithm for deciding the satisfiability of a logical formula to be *sound* and *complete*.

[6]

b) Explain how well-formed formulas of propositional logic can be represented by Prolog terms.

Specify an algorithm for a tautology checker which could be implemented in Prolog, whereby querying the checker succeeds if a given term is a tautology. (Only the steps in the algorithm are required, not the Prolog code.)

Explain the relationship between the tautology checker and truth tables. Indicate the limitations of this algorithm for tautology checking in practice.

[6]

c) Consider the following English statements:

All cats are smarter than all dogs.
All dogs are smarter than all muskrats.
All cool animals are cats.
All dumb animals are dogs.
'Smarter than' is a transitive relation.
Doogle is a cool animal.
Boris is a dumb animal.
Elon is a muskrat.

Express these statements in first-order predicate logic, and convert them to a set of Horn clauses.

[4]

d) Prove, using resolution and proof by negation, that *Doogle is smarter than Elon*. The proof should show all the unifiers.

[4]

4. This question is about reasoning in Propositional Logic, Predicate Logic, and Propositional Modal Logic, using the **KE** calculus.

Important: any answer to any part that doesn't use line numbers and rule references to indicate how new formulas have been derived will not get full marks.

a) In the *Obligatio Game*, a finite number n of rounds is chosen, depending on the severity of the exam. The examiner then gives the candidate successive assertions, $\phi_1, \phi_2, \ldots, \phi_n$ that she or he has to either *accept* or *reject* as each one is put forward. In the former case, ϕ_i is added to the candidate's stock of commitments; in the latter case, the negation $\neg\phi_i$ is added.

The candidate passes the exam if she maintains the consistency of her stock of commitments throughout all n assertions.

Suppose the candidate is exposed (successively) to the following three ($n = 3$) assertions:

1. $p \vee q$
2. $\neg((p \wedge q) \rightarrow r)$
3. $(p \wedge q) \leftrightarrow r$

Using the **KE** calculus, show what sequences of answers would ensure that the student passes the exam.

(Hint: use **KE** for model building, for all combinations of *accept* or *reject* of the assertions. If the proof has at least one open branch, then this implies a consistent sequence of answers.)

[8]

b) Prove the following syllogism using the **KE** calculus for first-order Predicate Logic:

All martyrs are phlegmatic
Some coughers are martyrs
Therefore, some coughers are phlegmatic

[3]

c) Define a Kripke Model, and explain how it is used to give a semantics for formulas of Modal Logic.

[3]

d) Using the **KE** calculus for Propositional Modal Logic, prove that the characteristic formulas for serial, symmetry and transitivity relations are theorems of logic **S5**:

(*i*) $\Box P \rightarrow \Diamond P$
(*ii*) $P \rightarrow \Box\Diamond P$
(*iii*) $\Diamond P \rightarrow \Box\Diamond P$

[6]

5. This question is about search algorithms for 2-player games, and the Simplified Event Calculus (EC).

a) Explain the difference between the MiniMax, MiniMax to Fixed Ply, and AlphaBeta search algorithms for 2-player games.

[6]

b) Consider the graph shown in Figure 5.1. Suppose that the Minimax algorithm to 3-ply would produce the evaluation of the leaf nodes as shown.

Using the AlphaBeta algorithm, identify which branches would be pruned, and explain why.

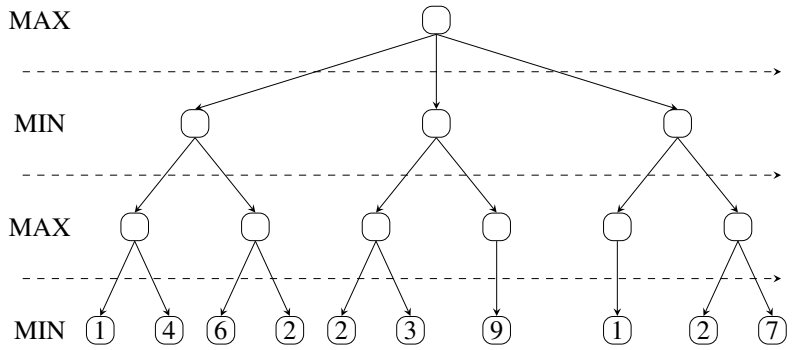


Figure 5.1 Hypothetical search space using MiniMax to fixed ply

[4]

c) The Blocks World scenario imagines a set of movable coloured blocks situated on a table, as illustrated in Figure 5.2.

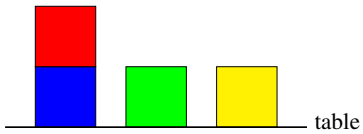


Figure 5.2 Blocks World: Initial State

There is a robot which can perform two actions: it can *stack* a block by taking any single block that is on the table and putting it on any other block, and it can *unstack* any block that is on another block. The goal is to build stacks of blocks called *towers* in a specified arrangement of colours.

Making any reasonable assumptions, specify the initial state of the Blocks World using `initially/1` facts about an identified multi-valued fluent `isOn`, whose value range will be `table` if the block is on the table, or otherwise another block.

Define `initiates/3` axioms of the Simplified EC for reasoning about the effects of *stack* and *unstack* actions. The actions should be valid (i.e., cannot unstack a block that is on the table, can only stack a block that is on the table).

Write a Prolog predicate that, given a block `B` and a time `T`, succeeds if the block `B` is the top block of a tower greater than height 1, and computes the tower itself.

[10]